

Universitatea POLITEHNICA București

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Proiect 2 – Aplicații practice ale microcontrolerelor

Tema 25 – Sistem de control al motorului utilizând STM32, ESP32 și MPU6050

Coordonator proiect: Guzu Nicolae-Alexandru

Titular disciplină: Guzu Nicolae-Alexandru

Realizat de: Jugănaru Elena-Alexandra, Nedelcu Alexia-Ioana, Zanfira Dan-Constantin

Grupa: 434A

An universitar : 2025-2026

Cuprins

1. Introducere:	
1.1 Prezentarea proiectului	3
1.2 Obiectivele proiectului	3
2. Resurse hardware:	
2.1 STM32 NUCLEO-L476RG	3
2.2 ESP32 Dev Module	4
2.3 Senzor MPU6050	4
2.4 Motor DC	4
2.5 Driver motor	4
2.6 Breadboard și conexiuni	4
3. Resurse software:	
3.1 Arduino IDE.....	5
3.2 STM32CubeIDE	5
3.3 STM32CubeMX	5
4. Implementare hardware:	
4.1 Principiul de funcționare.....	5
4.2 Comunicatia I2C.....	7
4.3 Comunicația UART	7
4.4 Generarea PWM	8
4.5 Controlul direcției	8
5. Implementare software:	
5.1 Algoritmul ESP32	8
5.2 Organigrama codului ESP32	9
5.3 Explicația codului Arduino	10
5.4 Algoritmul aplicației STM32	13
5.5 Diagrama FSM	13
5.6 Protocolul de comunicare	14
6. Testare și validare:	
6.1 Testarea comunicației I2C	15
6.2 Testarea comunicației UART	15
6.3 Testarea controlului motorului	15
6.4 Probleme întâlnite	15
7. Concluzii	16
8. Bibliografie	16
BOM	17

1. Introducere

1.1 Prezentarea proiectului

Proiectul realizat constă într-un sistem embedded pentru controlul unui motor DC utilizând un microcontroler STM32 NUCLEO-L476RG împreună cu un modul ESP32 și un senzor MPU6050.

Sistemul permite controlul direcției și vitezei motorului în funcție de înclinarea senzorului MPU6050. Datele de orientare sunt preluate de ESP32 prin interfața I2C, procesate și transmise către STM32 prin comunicație serială UART.

STM32 interpretează comenzile primite și generează semnale PWM pentru controlul motorului.

Proiectul demonstrează integrarea mai multor periferice hardware și software:

- comunicație I2C;
- comunicație UART;
- generare PWM;
- control motor DC;
- achiziție date senzori;
- procesare date în timp real.

1.2 Obiectivele proiectului

Obiectivele principale ale proiectului sunt:

- realizarea unei comunicații stabile între ESP32 și STM32;
 - achiziția datelor de accelerație de la MPU6050;
 - calcularea unghiului de înclinare;
 - transmiterea comenzilor către STM32;
 - controlul direcției și vitezei motorului folosind PWM;
 - implementarea unei arhitecturi embedded funcționale;
 - testarea și validarea funcționării sistemului.
-

2. Resurse hardware

2.1 STM32 NUCLEO-L476RG

STM32 NUCLEO-L476RG este placa principală utilizată pentru controlul motorului. Aceasta folosește microcontrolerul STM32L476RG bazat pe nucleul ARM Cortex-M4.

Caracteristici principale:

- frecvență maximă: 80 MHz;

- memorie Flash: 1 MB;
- SRAM: 128 KB;
- periferice UART, I2C, TIMERE și PWM;
- alimentare la 3.3V.

Rolul STM32 în proiect este:

- recepția datelor seriale de la ESP32;
- interpretarea comenzilor;
- generarea semnalului PWM;
- controlul direcției motorului.

2.2 ESP32 Dev Module

ESP32 este utilizat pentru citirea datelor de la MPU6050 și transmiterea acestora către STM32.

Caracteristici principale:

- procesor dual-core;
- conectivitate Bluetooth și WiFi;
- interfață I2C;
- UART hardware;
- frecvență de până la 240 MHz.

Rolul ESP32:

- comunicare cu MPU6050 prin I2C;
- procesarea valorilor senzorului;
- calculul unghiului de înclinare;
- transmiterea datelor prin UART.

2.3 Senzor MPU6050

MPU6050 este un senzor care conține:

- accelerometru pe 3 axe;
- giroscop pe 3 axe.

Comunicarea se realizează prin protocolul I2C.

În cadrul proiectului este utilizat accelerometrul pentru determinarea unghiului de înclinare.

2.4 Motor DC

Motorul DC reprezintă actuatorul principal al sistemului.

Controlul acestuia se realizează prin:

- modificarea direcției de rotație;
- modificarea factorului de umplere PWM.

2.5 Driver motor

Pentru alimentarea motorului a fost utilizat un driver de motor capabil să suporte curentul necesar funcționării.

Driverul permite:

- controlul direcției;
- controlul vitezei prin PWM;
- izolarea microcontrolerului de sarcina motorului.

2.6 Breadboard și conexiuni

Circuitul a fost realizat pe breadboard utilizând fire de conexiune.

Conexiunile principale sunt:

Componentă	Interfață	Pini utilizați
MPU6050 – ESP32	I2C	GPIO21 (SDA), GPIO22 (SCL)
ESP32 – STM32	UART	TX ↔ RX
STM32 – Driver motor	GPIO + PWM	TIM3 CH1

2.1 Conexiuni principale

3. Resurse software

3.1 Arduino IDE

Arduino IDE a fost utilizat pentru dezvoltarea aplicației care rulează pe ESP32.

Biblioteci utilizate:

- Wire.h
- Adafruit_MPU6050.h
- Adafruit_Sensor.h
- math.h

3.2 STM32CubeIDE

STM32CubeIDE a fost utilizat pentru configurarea și programarea microcontrolerului STM32.

Au fost utilizate următoarele periferice:

- UART4;
- TIM3 PWM;
- GPIO;
- I2C1.

3.3 STM32CubeMX

STM32CubeMX a fost utilizat pentru:

- configurarea pinilor;
- generarea codului de inițializare;
- configurarea perifericelor hardware.

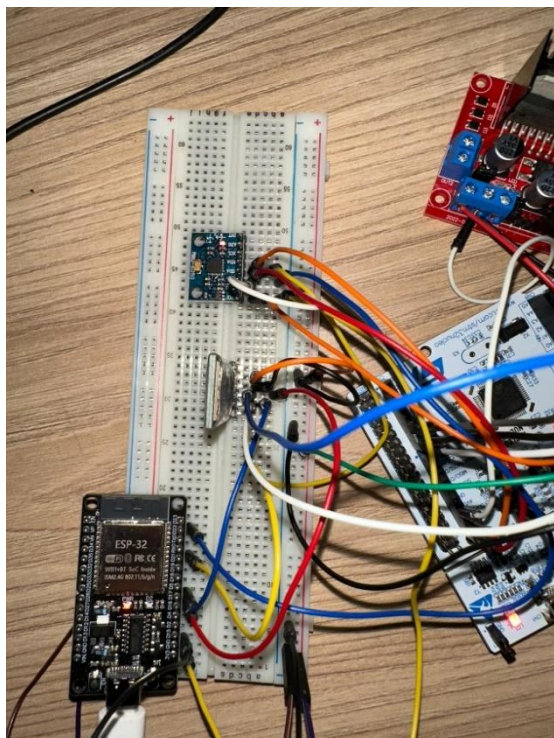
4. Implementare hardware

4.1 Principiul de funcționare

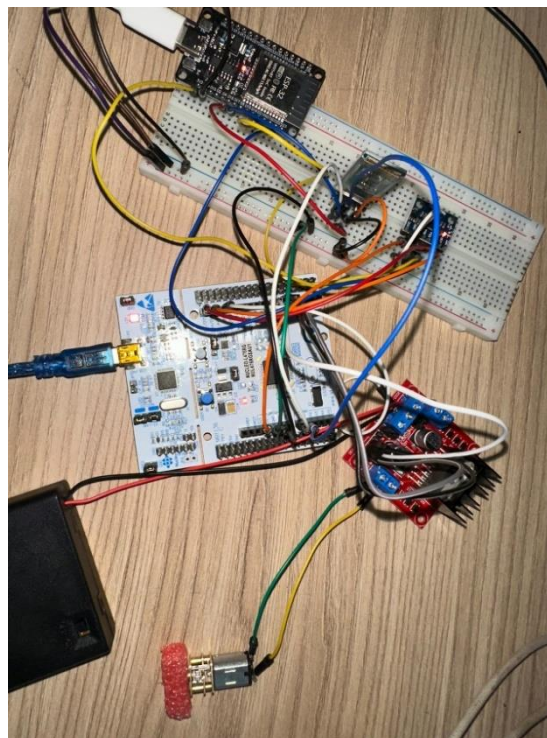
Sistemul funcționează după următorul principiu:

1. MPU6050 măsoară accelerația pe axe.
2. ESP32 citește valorile prin I2C.
3. ESP32 calculează unghiul de înclinare.
4. Unghiul este transmis prin UART către STM32.
5. STM32 interpretează valoarea primită.

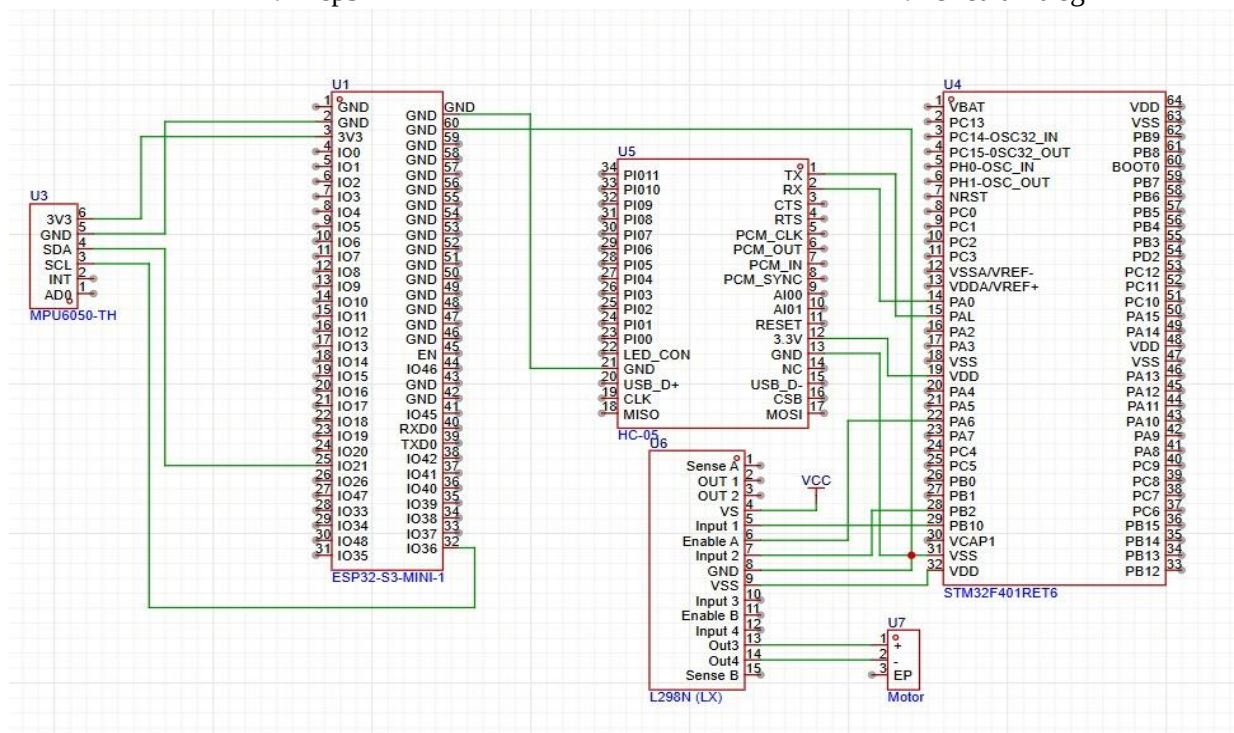
6. STM32 generează semnal PWM pentru controlul motorului.
7. Motorul își modifică viteza și direcția.



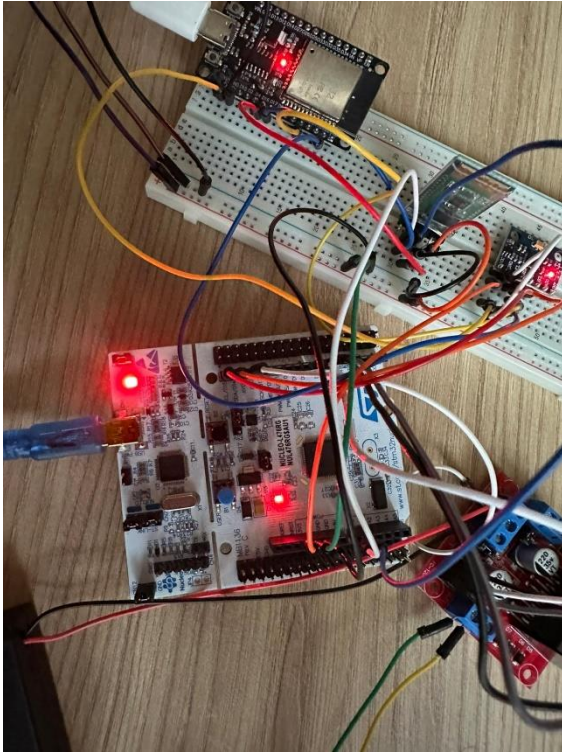
4.1 Esp32



4.2 circuit întreg



4.3 schema conexiuni



4.4 STM32

4.2 Comunicația I2C

Comunicația dintre ESP32 și MPU6050 se realizează prin magistrala I2C.

Pinii utilizați:

- SDA → GPIO21
- SCL → GPIO22

Frecvența comunicației este standard I2C.

4.3 Comunicația UART

Comunicarea dintre ESP32 și STM32 se realizează serial prin UART.

Parametri utilizați:

- baud rate: 9600;
- 8 biți de date;

- fără paritate;
- 1 bit stop.

4.4 Generarea PWM

STM32 utilizează perifericul TIM3 pentru generarea semnalului PWM.

Caracteristici:

- perioadă: 1000;
- control duty cycle;
- controlul vitezei motorului.

4.5 Controlul direcției

Direcția motorului este controlată utilizând doi pini GPIO:

- PA8;
- PB10.

În funcție de semnalul primit:

- unghi pozitiv → rotație înainte;
 - unghi negativ → rotație înapoi;
 - unghi zero → motor oprit.
-

5. Implementare Software

5.1 Algoritmul ESP32

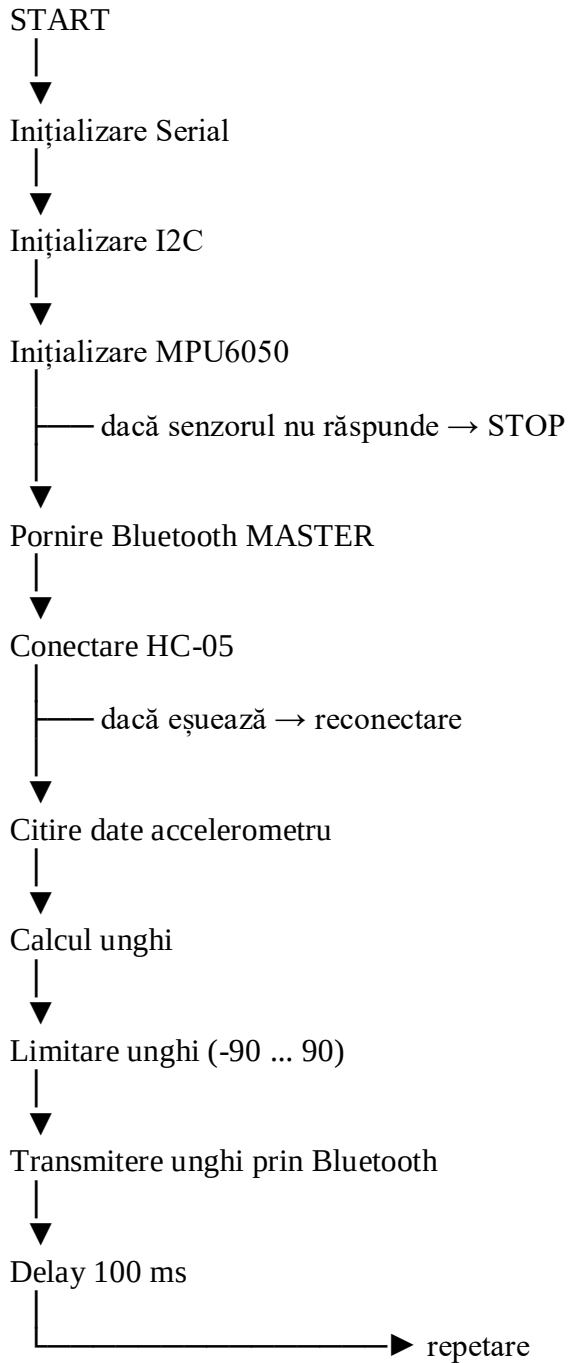
Programul dezvoltat în Arduino IDE rulează pe microcontrolerul ESP32 și are rolul de a:

- inițializa comunicația I2C;
- comunica cu senzorul MPU6050;
- calcula unghiul de înclinare;
- realiza conexiunea Bluetooth cu modulul HC-05;
- transmite datele către STM32.

Fluxul de funcționare al aplicației este următorul:

1. Inițializarea comunicației seriale pentru debugging;
2. Inițializarea magistralei I2C;
3. Detectarea și configurarea senzorului MPU6050;
4. Pornirea Bluetooth-ului ESP32 în modul MASTER;
5. Conectarea la modulul HC-05 prin adresa MAC;
6. Citirea valorilor accelerometrului;
7. Calcularea unghiului de înclinare;
8. Limitarea valorilor între -90° și 90° ;
9. Transmiterea valorii către STM32;
10. Repetarea continuă a procesului.

5.2 Organigrama codului ESP32



5.3 Explicația codului Arduino

Bibliotecile utilizate:

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include "BluetoothSerial.h"
#include <math.h>
```

5.1 biblioteci

Bibliotecile utilizate au următoarele roluri:

- Wire.h – comunicarea I2C;
- Adafruit_MPU6050.h – controlul senzorului MPU6050;
- Adafruit_Sensor.h – interfață standard pentru senzori;
- BluetoothSerial.h – comunicația Bluetooth Classic;
- math.h – funcții matematice.

Definirea obiectelor și pinilor

```
Adafruit_MPU6050 mpu;
BluetoothSerial SerialBT;

#define I2C_SDA 21
#define I2C_SCL 22
```

5.2 pini

Se creează obiectele necesare pentru:

- comunicarea cu MPU6050;
- transmiterea Bluetooth.

Pinii GPIO21 și GPIO22 sunt utilizați pentru magistrala I2C.

Configurarea adresei MAC

```
// Adresa MAC unică a modulului tău HC-05
uint8_t address[6] = {0x98, 0xDA, 0x50, 0x04, 0x13, 0x3A};
```

5.3 adresa MAC

ESP32 se conectează direct la modulul HC-05 utilizând adresa MAC unică a acestuia.

Această metodă oferă:

- conectare mai rapidă;
- stabilitate mai bună;
- eliminarea căutării automate.

Funcția `setup()` - rulează o singură dată la pornirea sistemului.

```
void setup(void) {
  Serial.begin(115200);
  while (!Serial) delay(10);

  Serial.println("=====");
  Serial.println("Inițializare sistem ESP32 Master.");
  Serial.println("=====");

  Wire.begin(I2C_SDA, I2C_SCL);

  // Verificare MPU6050
  if (!mpu.begin()) {
    Serial.println("❌ Nu s-a găsit MPU6050! Verifică firele (21=SDA, 22=SCL).");
    while (1) { delay(10); }
  }
  Serial.println("✅ MPU6050 a fost găsit!");

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  // Pornim Bluetooth-ul ca MASTER
  SerialBT.begin("ESP32_Master", true);
  Serial.println("✅ Bluetooth pornit în modul MASTER.");

  // Setăm PIN-ul de securitate descoperit
  SerialBT.setPin("1234", 4);

  Serial.println("Încerc conectarea directă la adresa MAC a HC-05...");

  // Conectare directă prin adresa MAC
  connected = SerialBT.connect(address);

  if(connected) {
    Serial.println("✅ CONECTAT CU SUCCES la HC-05!");
  } else {
    Serial.println("⚠️ Eșec la prima conectare. Reîncercăm automat în loop...");
  }
}
```

5.4 setup()

În etapa de inițializare sunt configurați parametrii principali ai senzorului MPU6050 pentru a obține măsurători stabile și precise ale mișcării. Sunt definite domeniile de funcționare pentru accelerometru și giroscop, astfel încât senzorul să poată detecta corect variațiile de poziție și înclinare ale sistemului. De asemenea, este activat un filtru digital intern care reduce zgomotul și fluctuațiile nedorite ale semnalelor provenite de la senzor, contribuind la stabilitatea valorilor utilizate în calculul unghiului.

După configurarea senzorului este inițializată comunicația Bluetooth a modului ESP32. Acesta funcționează în modul MASTER, având rolul de a iniția și menține conexiunea cu modulul HC-05 conectat la sistemul bazat pe STM32. Pentru autentificare și securizarea legăturii wireless este configurat un cod PIN, iar conectarea se realizează direct utilizând adresa MAC unică a modului HC-05. Folosirea adresei MAC permite evitarea procesului de căutare automată a dispozitivelor Bluetooth și asigură o conectare mai rapidă și mai stabilă între cele două sisteme.

În urma realizării conexiunii, ESP32 poate transmite în timp real valorile calculate ale unghiului către STM32, care le utilizează pentru controlul motorului DC.

Funcția `loop()` - rulează continuu.

```
void loop() {  
  // Logica de reconectare automată dacă se pierde semnalul  
  if(!connected) {  
    Serial.println("❌ Încerc reconectarea directă la HC-05...");  
    connected = SerialBT.connect(address);  
  
    if(connected) {  
      Serial.println("✅ Conectat cu succes la HC-05!");  
    } else {  
      Serial.println("❌ Conexiune eșuată. Reîncerc în 5 secunde...");  
      delay(5000);  
      return;  
    }  
  }  
  
  // Citirea senzorului și trimiterea datelor  
  sensors_event_t a, g, temp;  
  mpu.getEvent(&a, &g, &temp);  
  
  float ax = a.acceleration.x;  
  float ay = a.acceleration.y;  
  float az = a.acceleration.z;  
  
  // Calculul unghiului  
  int unghi = (atan2(ax, sqrt(ay*ay + az*az)) * 180.0) / PI;  
  
  if(unghi > 90) unghi = 90;  
  if(unghi < -90) unghi = -90;  
  
  // Trimitem unghiul prin Bluetooth către HC-05  
  SerialBT.println(unghi);  
  
  // Afișăm și în Serial Monitor pentru verificare  
  Serial.print("Unghi trimis: ");  
  Serial.println(unghi);  
  
  delay(100);  
}
```

5.5 `loop()`

Funcția `loop()` reprezintă secțiunea principală de execuție a programului și rulează continuu pe durata funcționării sistemului. În această etapă este verificată permanent starea conexiunii Bluetooth dintre ESP32 și modulul HC-05. În cazul în care conexiunea este pierdută, sistemul încearcă automat reconectarea, asigurând astfel continuitatea comunicației dintre dispozitive fără intervenția utilizatorului.

După confirmarea conexiunii, ESP32 realizează citirea valorilor furnizate de senzorul MPU6050. Sunt preluate datele de accelerație și rotație de pe cele trei axe, iar pe baza acestora este calculat unghiul de înclinare al sistemului. Calculul este realizat utilizând funcții trigonometrice, rezultatul fiind convertit în grade pentru a putea fi utilizat mai ușor în controlul motorului.

Pentru a evita apariția valorilor instabile sau depășirea limitelor mecanice ale sistemului, unghiul obținut este limitat în intervalul -90° și $+90^\circ$. Valoarea finală este transmisă prin Bluetooth către modulul HC-05, care o redirecționează către microcontrolerul STM32.

În paralel, valorile calculate sunt afișate și în Serial Monitor pentru debugging și verificarea funcționării sistemului în timp real. Procesul este repetat periodic, la un interval de aproximativ 100 ms, pentru a asigura un control fluid și rapid al motorului.

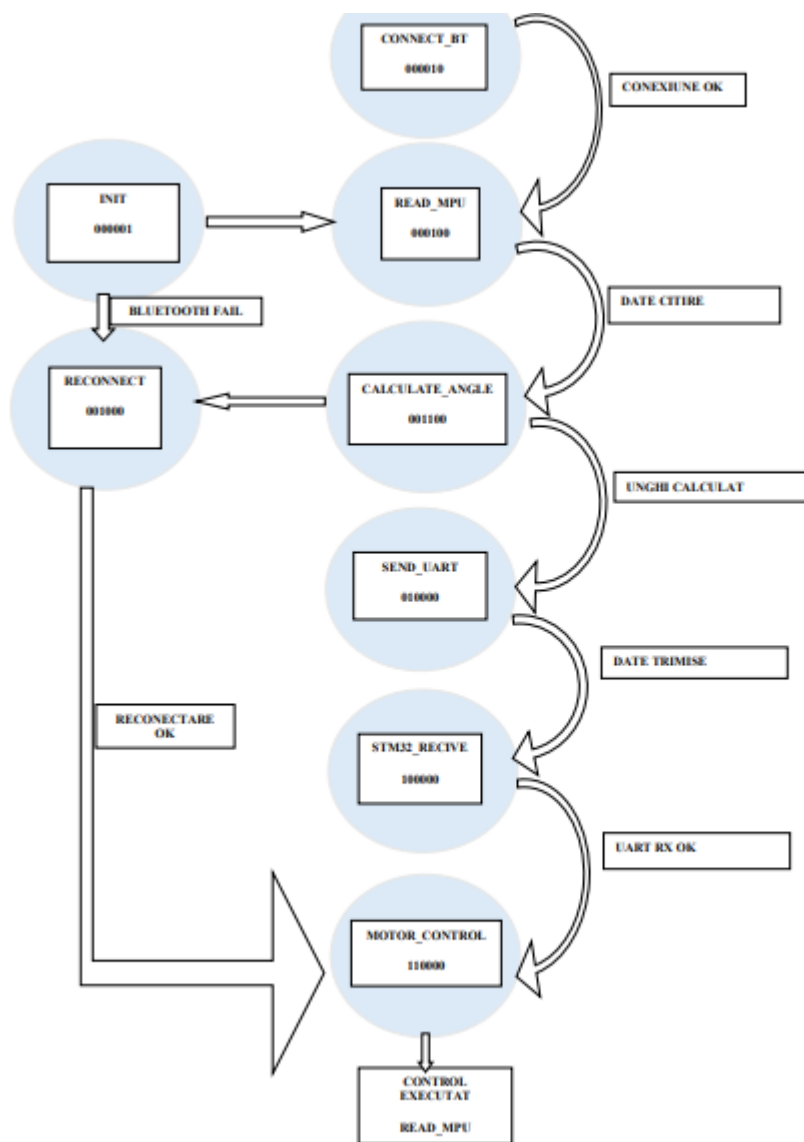
5.4 Algoritmul aplicației STM32

Programul dezvoltat în STM32CubeIDE are rolul de a primi datele transmise de ESP32 prin UART și de a controla motorul DC utilizând semnale PWM și pini GPIO pentru direcție.

Fluxul general de funcționare este următorul:

1. Inițializarea perifericelor STM32;
2. Configurarea UART pentru recepția datelor;
3. Pornirea semnalului PWM;
4. Recepția caracterelor prin întreruperi UART;
5. Formarea valorii complete a unghiului;
6. Conversia textului în valoare numerică;
7. Calcularea factorului de umplere PWM;
8. Stabilirea direcției de rotație;
9. Controlul motorului;
10. Repetarea continuă a procesului

5.5 Diagrama FSM



5.6 FSM

5.6 Protocolul de comunicare

Protocolul de comunicație implementat în cadrul proiectului este utilizat pentru transmiterea valorilor unghiului de înclinare între modulul ESP32 și microcontrolerul STM32 prin intermediul conexiunii Bluetooth realizate cu ajutorul modulului HC-05.

Comunicarea se realizează utilizând transmisie serială UART, datele fiind trimise sub formă de caractere ASCII. ESP32 calculează periodic unghiul de înclinare pe baza valorilor furnizate de senzorul MPU6050 și transmite această valoare către STM32 prin conexiunea Bluetooth.

Parametrii protocolului

Parametru	Valoare
Tip comunicație	UART prin Bluetooth
Modul Bluetooth	HC-05

Baudrate	9600 bps
Format date	ASCII
Delimitator mesaj	\n
Domeniu valori	-90° ... +90°
Frecvență transmitere	10 Hz

Structura mesajului transmis

<unghi>\n

ESP32 transmite periodic valorile unghiului calculate pe baza datelor provenite de la senzorul MPU6050. STM32 primește mesajele prin UART4 utilizând întreruperi, ceea ce permite recepția continuă a datelor fără blocarea execuției programului principal.

După recepționarea unei valori complete, STM32 calculează semnalul PWM necesar controlului vitezei motorului și stabilește direcția de rotație în funcție de semnul unghiului primit.

6. Testare și validare

6.1 Testarea comunicației I2C

A fost verificată comunicarea dintre ESP32 și MPU6050.

Rezultat:

- senzor detectat corect;
- valorile accelerației au fost citite cu succes.

6.2 Testarea comunicației UART

A fost verificată transmiterea datelor dintre ESP32 și STM32.

Rezultat:

- valorile au fost transmise corect;
- nu au fost observate erori de transmisie.

6.3 Testarea controlului motorului

S-au realizat teste pentru:

- rotație înainte;
- rotație înapoi;
- oprire;
- variația vitezei.

Rezultat:

- motorul răspunde corect la comenzile primite.

6.4 Probleme întâlnite

În timpul dezvoltării au fost întâlnite următoarele probleme:

- configurarea comunicației UART;
- conectarea ESP32 la calculator;
- identificarea porturilor COM;
- configurarea perifericelor în STM32CubeMX;

- probleme de conectare ST-LINK.

Problemele au fost rezolvate prin:

- utilizarea unui cablu USB de date;
 - reconfigurarea pinilor UART;
 - verificarea conexiunilor hardware;
 - reconfigurarea debuggerului.
-

7. Concluzii

În cadrul acestui proiect a fost realizat un sistem embedded complet pentru controlul unui motor DC utilizând STM32, ESP32 și senzorul MPU6050.

Proiectul a demonstrat:

- utilizarea comunicației I2C;
- utilizarea comunicației UART;
- generarea semnalelor PWM;
- controlul unui actuator;
- integrarea mai multor platforme embedded.

Sistemul funcționează în timp real și răspunde corect la modificarea poziției senzorului.

Posibile îmbunătățiri:

- implementarea unui control PID;
 - utilizarea Bluetooth pentru control wireless;
 - afișarea parametrilor pe LCD;
 - filtrarea avansată a datelor de la senzor.
-

8. Bibliografie

- [1] Documentație oficială STM32L476RG, STMicroelectronics.
- [2] Documentație ESP32, Espressif Systems.
- [3] MPU6050 Datasheet, InvenSense.
- [4] <https://www.st.com/>
- [5] <https://docs.espressif.com/>
- [6] https://github.com/adafruit/Adafruit_MPU6050
- [7] <https://www.arduino.cc/>
- [8] <https://www.st.com/en/development-tools/stm32cubeide.html>

BOM

Nr	Componentă	Model	Cantitate	Rol în proiect	Link
1	Placă dezvoltare STM32	STM32 NUCLEO-L476RG	1	Control motor și generare PWM	oferit de facultate
2	Placă dezvoltare ESP32	ESP32 DevKit V1	1	Achiziție date IMU și transmisie Bluetooth	oferit de facultate
3	Senzor IMU	MPU6050	1	Măsurarea unghiului mâinii	MPU6050 Accelerometer and Gyroscope Module (soldered pins)
4	Modul Bluetooth	HC-05 Master/Slave	1	Comunicație wireless UART	Modul Bluetooth master slave HC-05 cu adaptor (compatibil 3.3 V si 5 V), 6 pini - eMAG.ro
5	Driver motor	L298N	1	Control direcție și viteză motor	Modul cu Driver de Motoare Dual L298N Rosu motor DC Arduino / PIC / AVR / ARM / STM32 Dual H Bridge Stepper Drive Control
6	Motor DC	Motor cu reductie, GA12 N20, 6V, 200RPM	1	Actuator mecanic	Motor cu reductie, GA12 N20, 6V, 200RPM - eMAG.ro
7	Kit Breadboard+Fire conexiune	MB-102+Jumper wires	1 set	Proiectare și realizarea conexiunilor electrice	Kit Plusivo pentru Introducere în Electronică
8	Cablu USB	USB data cable	2	Programare și alimentare	N/A
9	Suport baterii	Suport 4xAA	1	Alimentare motor și driver L298N	Carcasă pentru Baterii (4 x R6)
10	Baterii AA	Baterii alcaline AA 1.5V	4	Alimentare sursă externă	N/A